

UNIT2

Q1:What is the difference between a key and a super key? Explain with an example. Also, discuss the various reasons that lead to the occurrence of NULL values in relations.

S.NO Super Key	Primary Key
1. Super Key is an attribute (or set of attributes) that is used to uniquely identifies all attributes in a relation.	Primary Key is a minimal set of attribute (or set of attributes) that is used to uniquely identifies all attributes in a relation.
2. All super keys can't be primary keys.	Primary key is a minimal super key.
3. Various super keys together makes the criteria to select the candidate keys.	We can choose any of the minimal candidate key to be a primary key.
4. In a relation, number of super keys are more than number of primary keys.	While in a relation, number of primary keys are less than number of super keys.
5. Super key's attributes can contain NULL values.	Primary key's attributes cannot contain NULL values.

Null Values Absence of a data item is represented by a special value called null value. The desired reasons that lead to the occurrence of null values in relations are as given below:

When value of an attribute is not applicable for a tuple. it can be marked as NULL.
When value for an attribute unknown for a tuple, it can be marked as NULL.

An important concept is that of nulls, which are used to represent the values of attributes that may be unknown or may not apply to a tuple. A special value, called null, is used for these cases. In general, we can have *several meanings* for null values, such as "value unknown," "value exists but is not available," or "attribute does not apply to this tuple." It is possible to devise different codes for different meanings of null values. Incorporating different types of null values into the relational model operations has proven difficult and is outside the scope of our presentation.

Q2:Define union compatibility of relations. Explain the set theoretical operations in relational algebra with example.

Two relations R and S , are said to be UNION COMPATIBLE, only if they satisfy these two conditions:

1. Both the relations , i.e. R and S should be of the same arity, i.e. they should have the same number of attributes.
2. Domain of the similar attributes, i.e. domain of ith attribute of R and domain of ith attribute of S, should be same and this is applicable to all the attributes.

The set operation are mainly categorized into the following:

- Union operation($\cup$)
 - Intersection operation($\cap$)
 - Set difference or Minus operation($-$)
 - Cartesian Product($\times$)
-
- UNION Operation , Binary operation, denoted by $\cup$
 - The result of $R \cup S$, is a relation that includes all tuples that are either in R or in S or in both R and S
 - Duplicate tuples are eliminated
 - The two operand relations R and S must be “type compatible” (or UNION compatible)
 -
 - INTERSECTION OPERATION
 - Is denoted by $\cap$ The result of the operation $R \cap S$, is a relation that includes all tuples that are in **both R and S** The attribute names in the result will be the same as the attribute names in R The two operand relations R and S must be “type compatible”
 - SET DIFFERENCE
 - (also called MINUS or EXCEPT) is denoted by $-$ The result of $R - S$, is a relation that includes all tuples that are in R but not in S. The attribute names in the result will be the same as the attribute names in R The two operand relations R and S must be “type compatible”

Cartesian Product:

- - **Cartesian Product in DBMS** is an operation used to merge columns from two relations.
 - It is denoted by ($\times$)

Example:

Consider a relation Student(FIRST, LAST) and Faculty(FIRSTN, LASTN) given below :

First	Last
Aisha	Arora
Bikash	Dutta
Makku	Singh
Raju	Chopra
◀	

FirstN	LastN
Raj	Kumar
Honey	Chand
Makku	Singh
Karan	Rao
◀	

1. Student UNION Faculty :

Student U Faculty

First	Last
Aisha	Arora
Bikash	Dutta
Makku	Singh
Raju	Chopra
Raj	Kumar
Honey	Chand
Karan	Rao

2. Student INTERSECTION Faculty :

Student ∩ Faculty

First	Last
Makku	Singh

3. Student MINUS Faculty :

Student - Faculty

First	Last
Aisha	Arora
Bikash	Dutta
Raju	Chopra
Raj	Kumar
Honey	Chand

A:	<table><tr><th>a1</th><th>a2</th></tr><tr><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td></tr><tr><td>5</td><td>6</td></tr></table>	a1	a2	1	2	3	4	5	6	B:	<table><tr><th>b1</th></tr><tr><td>7</td></tr><tr><td>8</td></tr></table>	b1	7	8	A × B	⇒	<table><tr><th>a1</th><th>a2</th><th>b1</th></tr><tr><td>1</td><td>2</td><td>7</td></tr><tr><td>1</td><td>2</td><td>8</td></tr><tr><td>3</td><td>4</td><td>7</td></tr><tr><td>3</td><td>4</td><td>8</td></tr><tr><td>5</td><td>6</td><td>7</td></tr><tr><td>5</td><td>6</td><td>8</td></tr></table>	a1	a2	b1	1	2	7	1	2	8	3	4	7	3	4	8	5	6	7	5	6	8
a1	a2																																					
1	2																																					
3	4																																					
5	6																																					
b1																																						
7																																						
8																																						
a1	a2	b1																																				
1	2	7																																				
1	2	8																																				
3	4	7																																				
3	4	8																																				
5	6	7																																				
5	6	8																																				

Q3: Explain in detail about various constraints used in database system [link](#)

Constraints in the databases can be categorized into 3 main categories:

1. Constraints that are applied in the data model is called **Implicit constraints**.
2. Constraints that are directly applied in the schemas of the data model, by specifying them in the **DDL(Data Definition Language)**. These are called as **schema-based constraints or Explicit constraints**.
3. Constraints that cannot be directly applied in the schemas of the data model. We call these Application based or **semantic constraints**.

So here we will deal with **Implicit constraints**.

Mainly Constraints on the relational database are of 4 types:

1. Domain constraints
2. Key constraints
3. Entity Integrity constraints
4. Referential integrity constraints

Let discuss each of the above constraints in detail.

1. Domain constraints :

1. Every domain must contain atomic values(smallest indivisible units) it means composite and multi-valued attributes are not allowed.
2. We perform datatype check here, which means when we assign a data type to a column we limit the values that it can contain. Eg. If we assign the datatype of attribute age as int, we cant give it values other then int datatype.

Example:

EID	Name	Phone
01	Bikash Dutta	123456789 234456678

Explanation: In the above relation, Name is a composite attribute and Phone is a multi-values attribute, so it is violating domain constraint.

2. Key Constraints or Uniqueness Constraints :

1. These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
2. A relation can have multiple keys or candidate keys(minimal superkey), out of which we choose one of the keys as primary key, we don't have any restriction on choosing the primary key out of candidate keys, but it is suggested to go with the candidate key with less number of attributes.
3. Null values are not allowed in the primary key, hence Not Null constraint is also a part of key constraint.

Example:

EID	Name	Phone
01	Bikash	6000000009
02	Paul	9000090009
01	Tuhin	9234567892

Explanation: In the above table, EID is the primary key, and first and the last tuple has the same value in EID ie 01, so it is violating the key constraint.

3. Entity Integrity Constraints :

1. Entity Integrity constraints says that no primary key can take NULL value, since using primary key we identify each tuple uniquely in a relation.

Example:

EID	Name	Phone
01	Bikash	9000900099
02	Paul	600000009
NULL	Sony	9234567892

Explanation: In the above relation, EID is made primary key, and the primary key cant take NULL values but in the third tuple, the primary key is null, so it is a violating Entity Integrity constraints.

4. Referential Integrity Constraints :

1. The Referential integrity constraints is specified between two relations or tables and used to maintain the consistency among the tuples in two relations.

2. This constraint is enforced through foreign key, when an attribute in the foreign key of relation R1 have the same domain(s) as the primary key of relation R2, then the foreign key of R1 is said to reference or refer to the primary key of relation R2.

3. The values of the foreign key in a tuple of relation R1 can either take the values of the primary key for some tuple in relation R2, or can take NULL values, but can't be empty.

Example:

EID	Name	DNO
01	Divine	12
02	Dino	22
04	Vivian	14

DNO	Place
12	Jaipur
13	Mumbai
14	Delhi

Explanation: In the above, DNO of the first relation is the foreign key, and DNO in the second relation is the primary key. DNO = 22 in the foreign key of the first table is not allowed since DNO = 22 is not defined in the primary key of the second relation. Therefore, Referential integrity constraints is violated here

q) Explain Structural constraint and Participations constraints with example(imp)

ans)

Structural constraint

Participations constraint [link](#) partial and full Participations

Q) Explain the entity integrity and referential integrity constraints with examples

Refer Q3

Q: Illustrate Aggregate functions and Grouping in Relational algebra with examples.

These functions take a collection of values as input and return a single value as the result. Some of the Aggregate Functions are as follows :

SUM : Takes a collection of values and return the SUM of those values.

AVG : Takes a collection of values and return the AVERAGE of those values.

COUNT : Takes a collection of values and return the number of those values in the collection.

MIN, MAX : Takes a collection of values and return the MINIMUM and MAXIMUM values in a collection.

For example : Find the total sum of salaries of all Female Employees in an Organization.

Query : G (SUM(salary)) (Female_employees)

This will return the sum of salaries of female employees of the relation

Q: Discuss the characteristics of relations that make them different from ordinary tables and files.

Ordering of tuples in a relation $r(R)$:

□

The tuples are *not considered to be ordered*, even though they appear to be in the tabular form.

Ordering of attributes in a relation schema R (and of values within each tuple):

□

We will consider the attributes in $R(A_1, A_2, \dots, A_n)$ and the values in $t = \langle v_1, v_2, \dots, v_n \rangle$ to be ordered.

Values in a tuple:

All values are considered atomic (indivisible).

Each value in a tuple must be from the domain of the attribute for that column

■

If tuple $t = \langle v_1, v_2, \dots, v_n \rangle$ is a tuple (row) in the relation state r of $R(A_1, A_2, \dots, A_n)$

Then each v_i must be a value from $dom(A_i)$

Notation:

We refer to **component values** of a tuple t by:

$t[A_i]$ or $t.A_i$

■

This is the value v_i of attribute A_i for tuple t

Similarly, $t[A_u, A_v, \dots, A_w]$ refers to the subtuple of t containing the values of attributes $A_u, A_v, \dots, A_w$, respectively in t

Another answer:

Characteristics of Relational Database Model

Each relation in a database must have a distinct or unique name which would separate it from the other relations in a database.

- A relation must not have two attributes with the same name.

Duplicate tuples must not be present in a relation.

Q: The RENAME operation is used to rename the output of a relation. Sometimes it is simple and suitable to break a complicated sequence of operations and rename it as a relation with different names. Reasons to rename a relation can be many, like –

We may want to save the result of a relational algebra expression as a relation so that we can use it later.

- We may want to join a relation with itself, in that case, it becomes too confusing to specify which one of the tables we are talking about, in that case, we rename one of the tables and perform join operations on them.

- **Q: Discuss the division operation in relational algebra with an example**

Division operation

The division operator is used for queries which involve the 'all'.

$R1 \div R2$ = tuples of R1 associated with all tuples of R2.

Division: example

R		S	R ÷ S
Lecturer	Module	Subject	Lecturer
Brown	Compilers	Prolog	Green Lewis
Brown	Databases		
Green	Prolog		
Green	Databases		
Lewis	Prolog		
Smith	Databases		

Q. List and Explain the steps of an algorithm for ER to Relational Mapping with an example./

Step 1: Mapping of Regular Entity Types

Step 2: Mapping of Weak Entity Types

Step 3: Mapping of Binary 1:1 Relation Types

Step 4: Mapping of Binary 1:N Relationship Types.

Step 5: Mapping of Binary M:N Relationship Types.

Step 6: Mapping of Multivalued attributes.

Step 7: Mapping of N-ary Relationship Types.

(Explain in brief https://docs.google.com/presentation/d/1fPgU_9iWOyqjOIEvn2kVXHvVg6ooF4-/edit#slide=id.p4)

Q. Define a Join operation. What are the different types of JOIN operations in Relational algebra. Explain with an example. [link](#)

Q. Discuss the various update operations that can be performed on a relation. How are the constraint violations dealt with?

Define candidate key, super key and foreign key with examples

Discuss the various update operations that can be performed on a relation. How are the constraint violations dealt with

Define the terms with an example for each:

- i) Domain
 - ii) Relation schema
 - iii) Relation state
- Relation database schema.

Differentiate between two approaches of view implementation

With respect to the Relational data model, explain the concept of primary keys and Foreign keys. Illustrate your answer with examples

Q. Consider the schema given below:

Vehicle(reg-no, make, color)

Person (eno, name, address)

Owner(eno, reg-no)

Write relational algebra expressions for the following queries.

- i) List the names of persons who do not own car
- ii) List the names of persons who own only Hyundai cars
- ii) List the names of persons who own purple color cars
- iv) List the colors of vehicles grouped by make

Assume the following relations:

BOOKS(DocId, Title, Publisher, Year)

STUDENTS(StId, StName, Major, Age)

AUTHORS(AName, Address)

borrows(DocId, StId, Date)

has-written(DocId, AName)

describes(DocId, Keyword)

Write relational algebra expressions for the following queries.

- i. List the year and title of each book
- ii. List the names of all students who have borrowed a book and who are CS majors.
- iii. List the title of books written by the author 'Silberschatz'.

Consider the following tables:

EMPLOYEE (person-name, street, city)

WORKS (person-name, company-name, salary)

COMPANY (company-name, city)

MANAGES (person-name, manager-name)

Write Relational algebraic queries for the following

- i. Find the names of all employees who work for First Bank Corporation.
- ii. Find the names and cities of residence of all employees who work for FirstBank Corporation.
- iii. Find the names, street address, and cities of residence of all employees who work for First Bank Corporation and earn more than \$10,000 per annum.
- iv. Find the names of all employees in this database who live in the same city as the company for which they work.

Consider the following table:

employee(empno, name, office, age)

books(isbn, title, authors, publisher)

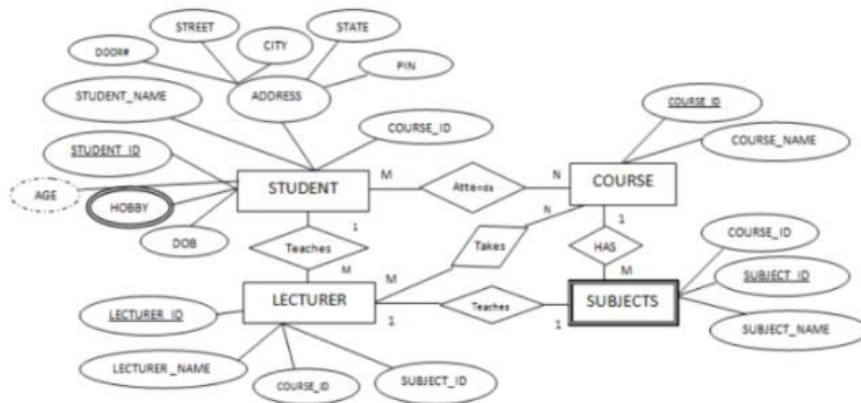
loan(empno, isbn, date)

Write Relational algebraic queries for the following

- i. Find the names of employees who have borrowed a book published by McGraw-Hill.
- ii. Find the names of employees who have borrowed all books published by McGraw-Hill.
- iii. Find the names of employees who have borrowed more than five different books published by McGraw-Hill.

different books published by McGraw-Hill.

- b) Explain the mapping of binary 1:N relationship and binary M:N relationship types of ER diagrams to relations. Convert the given ER diagram into relational schema. CO2 (08)



- c) Find the quotient for the following: $A/B_1, A/B_2$ and A/B_3 ; where CO2 (06)

- c) Find the quotient for the following: $A/B_1, A/B_2$ and A/B_3 ; where A, B_1, B_2 and B_3 are:

A	
Sno	Pno
S1	P1
S1	P2
S1	P3
S1	P4
S2	P1
S2	P2
S3	P2
S4	P2
S4	P4

B3
Pno
P1
P2
P4

B2
Pno
P2
P4

B1
pno
P2